Rickstore Web application penetration test

Samuel Rutherford

CMP319: Web Application Penetration Testing

BSc Ethical Hacking Year 3

2022/23

Note that Information contained in this document is for educational purposes.

.

Abstract

Rickstore groups sought out a cybersecurity specialist to perform a penetration test for their web application to develop an understanding of potential vulnerabilities present. The goals for the penetration test involved:

- Identifying numerous exploits which could harm the web application.
- Report the findings in a suitable format for someone of reasonable technical ability.

The exploits were performed using a virtualised Linux 'Kali' machine and given the assumed privilege of an average person using the internet to browse their online store. The penetration test was performed using recommendations from the OWASP methodology which ensured a sufficient and comprehensive test.

.

Contents

.

.

.

1 INTRODUCTION

1.1 BACKGROUND

As the business world evolves into using a digital landscape to market their products to consumers, web applications are necessary to present information and said products in a clear and concise format with e-commerce continuing to be on the rise.

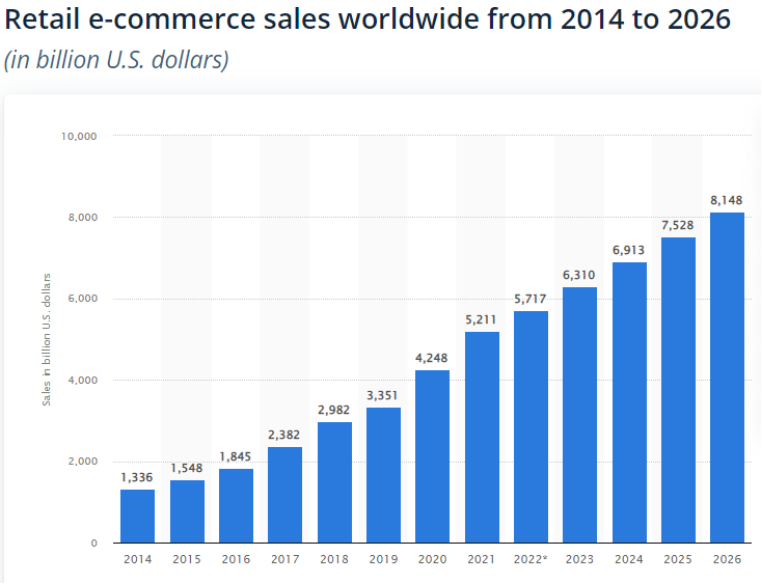


Fig 1.1 – Graph showing the projected rise in e-commerce sales globally from 2014 to 2026.

Web applications perfectly suit these needs as they are customisable, interactive, and incredibly adaptable to suit the business's needs. Additionally, web applications are also used by government infrastructure such as health services and informative pages. A downside to web applications is that they require technology, monitoring and administrators who are knowledgeable in computing. Commonly, one of those aspects are often missing leading to the web pages being vulnerable to a potentially cybersecurity attack. Malicious hackers will attempt to acquire information and disrupt the business or service through attacking the web application, causing an average of $4.35 million due to data breaches, a 2.6% increase from 2021. A suitable countermeasure to this is to hire a penetration tester who is familiar with the methods cybercriminals would use on their website and suggest patches for any vulnerabilities found on the web application.

1.2 AIMS

The aim was to perform a comprehensive web application penetration test for a website before producing a report documenting the chosen methodology, findings and an evaluation on the website and process.

During the penetration test, specific goals include:

- Evaluate critical enumeration methods the web application leaked information through.
- Identify notable misconfigurations.
- Achieve administrator credentials/authentication.
- Identify any authorisation errors.
- Locate XSS and SQL injection vulnerabilities.
- Evaluate how information is stored and if encryption is present.

Finally, a report is to be produced informing of the exploits and any recommendations for the website.

1.3 METHODOLOGY

The methodology being used is the OWASP Web Application Security testing. During the testing, not every section was tested or netted usable results, these sections will be skipped unless covering them is significant. It is worth mentioning that before adopting this methodology, some initial reconnaissance for the web application was performed at a relatively simple low level.

2 PROCEDURE AND RESULTS

2.1 OVERVIEW OF PROCEDURE

To begin the procedure, the IP address for the web application was given. From that, the web application can be accessed using a web browser. Web application penetration tools found within Kali were also implemented and used to perform the exploits. Once the website was accessed, initial reconnaissance took place to develop a better understanding of the web application before utilising the OWASP methodology. After finishing the penetration test, a graph was made for the severity of each category of vulnerability and how severe it is to the web application. Severity was based on how much damage it can cause to the web application and how easy it is to perform the exploit.

Fig 2.1 – Bar chart displaying severity of each category of vulnerability tested from a scale of 1 to 10 with 10 being the most severe and vulnerable.

2.2 INITIAL RECONNAISSANCE

Upon accessing the website by typing '192.168.1.10' into a search engine, the user is greeted with the home page for 'Rickstore', a website that specialises in selling electronic products. The website appeared to be ran by 2 administrators, with their names located in the /about.php directory. Within the footer, links to the social media for the company are found however they only lead back to the index page. The web application also features a login system for customers and administrators.

3 INFORMATION GATHERING

3.1 SEARCH ENGINE DISCOVERY RECONNAISSANCE FOR INFORMATION LEAKAGE

Beginning following the OWASP Web Application Security Testing Methodology, information gathering takes place to gather knowledge about the web application in order to perform exploits where possible.

A common starting point to begin information gathering on a web application is to investigate the robots.txt page. The purpose of robots.txt is a file that indicates to spidering robots to not access these files. In doing so, the files names are leaked.

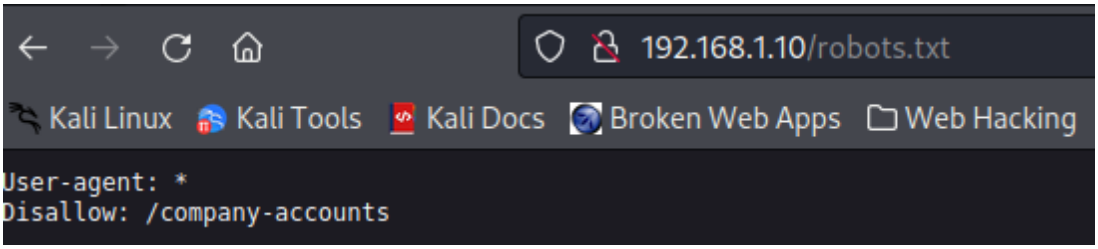


Fig 3.1 – robots.txt webpage for the application.

The leaked file name leads to a directory including a zipped folder full of Excel spreadsheets labelled 'fincances.zip' and another text document labelled 'readme.txt' which described the purpose of the zip folder, to contain the company's financial reports.

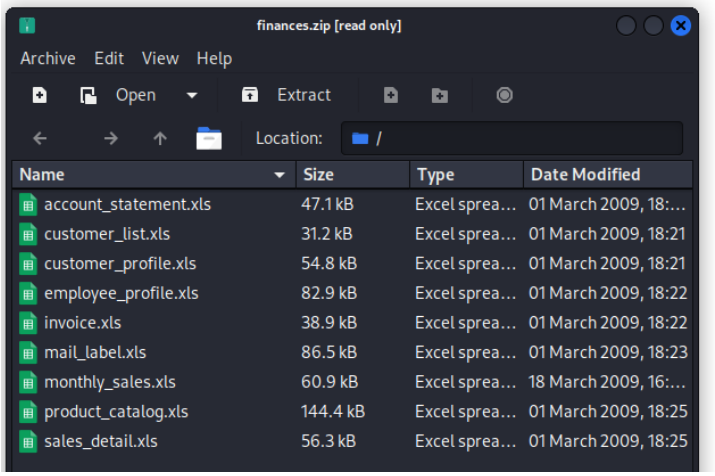


Fig 3.2 – Excel spreadsheets found within finances.zip.

3.2 FINGERPRINT WEB SERVER

Fingerprinting a web server can lead to essential information about the web application, this information is often used for further enumeration and performing exploits. To fingerprint the webserver, a HTTP request was captured using burpsuite.

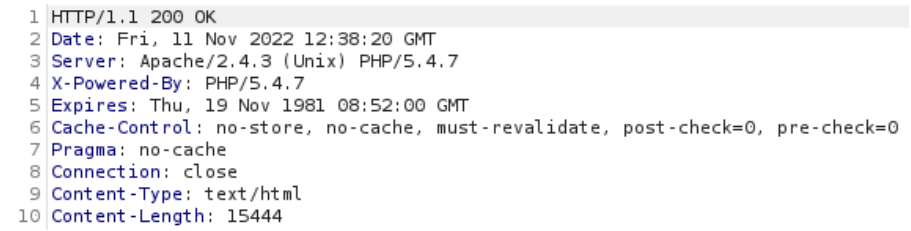


Fig 3.3 – HTTP packet captured using burpsuite.

The scanning tool known as Nikto also works. The following scan can be performed using the command 'nikto -h 192.168.1.10'. Full Nikto scan can be found in appendix B.

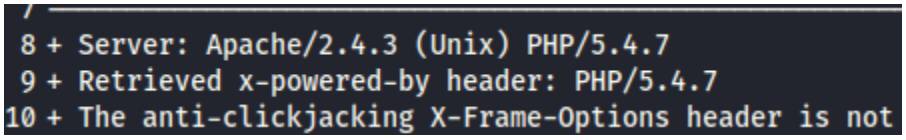
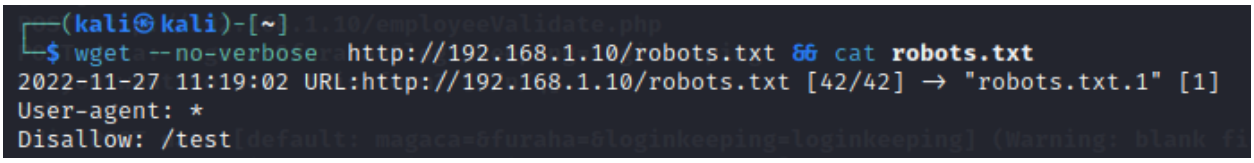


Fig 3.4 – Nikto scan for 192.168.1.10.

As seen from both results, the web server appears to be Apache/2.4.3 (Unix) PHP/5.4.7.

3.3 Review Webserver Metafiles for Information Leakage

Many web applications use metadata files which can be leaked to an attacker. The robots.txt file was already found and explored in section 3.1 however it is possible to find other metadata files. Using the Kali tool 'wget', it is possible to search web pages' source code. The following command displays the robots.txt page for 192.168.1.10 however it gave different results.



F.g 3.5 – Command used to view contents in robots.txt.

While the results from the command indicate that /test is another directory worth visiting, upon attempting to visit that page an error 404 code was received indicating that the page does not exist. Additionally, other directories recommended by OWASP were attempted such as /sitemap.xml, security.txt and humans.txt however they did not exist.

3.4 Enumerate Applications on Webserver

Using an NMAP scan it is possible to enumerate what applications are being ran on the webserver.

```
┌──(kali㉿kali)-[~]
└─$ nmap -sV 192.168.1.10
Starting Nmap 7.92 ( https://nmap.org ) at 2022-12-22 10:06 EST
Nmap scan report for 192.168.1.10
Host is up (0.00085s latency).
Not shown: 997 closed tcp ports (conn-refused)
PORT     STATE SERVICE VERSION
21/tcp   open  ftp     ProFTPD 1.3.4a
80/tcp   open  http    Apache httpd 2.4.3 ((Unix) PHP/5.4.7)
3306/tcp open  mysql   MySQL (unauthorized)
Service Info: OS: Unix

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.67 seconds
```

Fig 3.6 – NMAP scan for 192.168.1.10.

The command 'nmap -sV 192.168.1.10' operates by first inputting '-sV' to identify the operating system and service versions running and finally the IP address being enumerated, '192.168.1.10'. The services running is identified by packets being sent to the address and awaiting a response from the host. Depending on the packets received and from which port, the service can easily be identified, along with the version type. NMAP can accomplish by using a vast library of signatures within its database to recognise which ones match the signatures coming from the host.

3.5 Review Webpage Content for Information Leakage

Commonly within HTML/PHP documents, web developers can leave comments which leak information to attackers. Using Burpsuite web proxy, it is possible to review the HTTP response and view the HTML code:

```
18
19 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
20 <html lang="en-US" xmlns="http://www.w3.org/1999/xhtml" dir="ltr">
21   <head>
22     <title>
          RickStore Groups
       </title>
```

Fig 3.7 – Comment found within index.php detailing a source for part of the code.

The only comments found were ones crediting outside sources for use of code. Not every file and web page found within the web application was searched for information leakage due to time taken.

3.6 Map Execution Paths Through Application

Testing the application pathways helps attackers develop a thorough understanding of the web application. A method of achieving this is using automatic spidering, which can be utilised on OWASP ZAP. Due to the size of the output, the result is found within appendix A. Automatic spidering is the method a

deploying robots to automatically discover new URLs within the website. This is critical to identifying the size and scope of the website and any hidden previously missed web pages. This is also helpful in identifying the type of HTTP request.

3.7 Fingerprint Web Application Framework

The web application framework being used can often lead to specific exploits. Using tools such as burpsuite, NetCat and dirbuster, it was not possible to identify any commonly known web application frameworks.

3.8 Map Application Architecture

Breaking down the web applications architecture is essential to understanding how it operates. Some of this information was acquired during the prior steps.

- The web server was discovered to be Apache/2.4.3 (Unix) PHP/5.4.7.
- The web application has a web server.
- No static storage was found.
- There is at least one database present which has tables for employees, orders, customers, products, complaints/comments, and categories.
    - An attacker can make a confident guess about this information however once the web application has been compromised this information is confirmed.
    - The web application also uses MySQL.
- Authentication is achieved by the webpage 192.168.1.10/userValidate.php and uses either the employee or customer table to do so.
- Numerous third-party services/APIs are used:
    - Social media pages can be found within the footer (however they direct the user back to the home page).
    - The website uses numerous CSS files.
    - Uses a payment system.
- No evidence some security components currently:
    - Ports appear to be readily available and scannable.
    - Lacks an intrusion detection system.

4 Configuration and Deployment Management Testing

4.1 Test Network Infrastructure Configuration

Having correctly configured network infrastructure is crucial to maintaining security within it. Identifying vulnerabilities from the previously enumerated application architecture took place and possible vulnerability opportunities are elaborated upon.

Firstly, the web application could be vulnerable to SQL injection, to test this an OWASP ZAP automated scan was performed to identify any injection points.

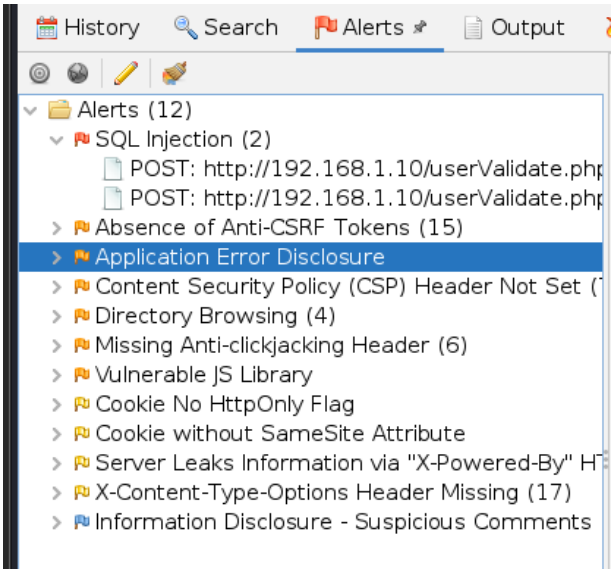


Fig 4.1 – SQL injection point identified on 192.168.1.10/userValidate.php during an OWASP ZAP automated scan.

An SQL injection point was found on the web page where authentication takes place. The only way to access this web page is when attempting to login or manually accessing it within the search bar.

4.2 Testing for Default Credentials

When a web application is misconfigured, often default credentials are present. This occurs when an administrator has failed to change the default username and password. The following usernames and passwords were attempted:

Username	Password	Successful
admin	admin	No
admin	password	No
owner	password	No

root	password	No

Table 4.2 – Table displaying the attempted username and password combinations.

4.3 Test File Extensions Handling for Sensitive Information

Often information can be leaked within hidden files and directories. A method of finding these files can be done using dirbuster. Dirbuster operates by using a word list and brute forcing common directories and file types onto the website.

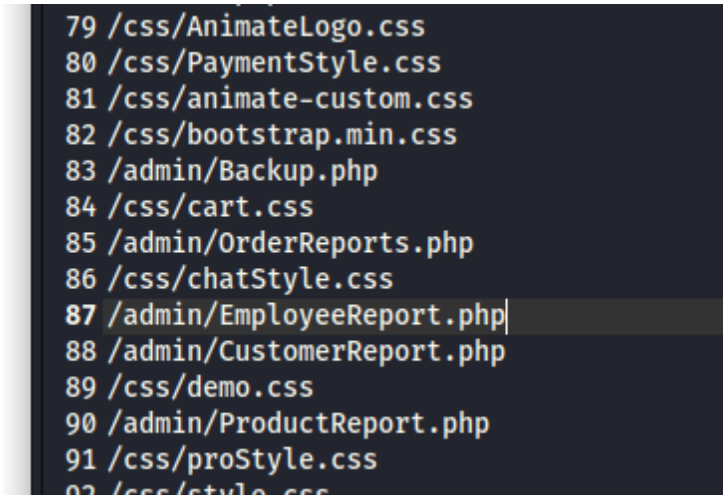


Fig 4.3 – Dirbuster report generated for 192.168.1.10.

From the dirbuster report, 2 particular files were identified containing valuable information. Normally files within the /admin sub directory return a 301-error indicating that they exist but require administrator authentication. However, /admin/EmployeeReport.php and /admin/CustomerReport.php do not require any authentication. CustomerReport.php can be found in appendix C and the entire dirbuster report can be found in appendix E.

SOMSTORE EMPLOYEE REPORS
EMPLOYEE REPORTS
Tuesday, November 08, 2022

Employee Record: 2

Employee ID	Employee Full Name	EMPLOYEE USER NAME
52	Mr Admin	admin
53	testadmin	testadmin

Fig 4.4 – Table including administrator usernames found at 192.168.1.10/admin/EmployeeReport.php.

The username for the administrator logins is identified within the table, these can be used for testing credentials later.

4.4 Enumerate Infrastructure and Application Admin Interfaces

The dirsbuter report generated in the previous section identified a hidden administrator subdirectory, /admin. For most of the files, authentication is required however some including sensitive information regarding administrators of the company can be accessed, such as /admin/EmployeeReport.php. Employee table can be found in appendix D.

4.5 Test HTTP Strict Transport Security

The HTTP strict transport security ensures all data and packets that travel between the web browser and server are encrypted. A method of testing this is searching a HTTP packet using curl:

```
┌──(kali㉿kali)-[~]
└─$ curl -s -D- http://192.168.1.10 | grep -i strict
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
```

Fig 4.5 – Using curl to search for the word 'strict' in a HTTP packet.

While the word strict was found within the packet, it was nothing to do with the security of the packet, indicating that there is no HTTP strict transport security.

4.6 Testing for Content Security Policy

Content security policy helps to protect web applications against Cross Site Scripting (XSS) and Clickjacking by restricting how resources such as JS and CSS files. Google has a tool online known as Google CSP evaluator:

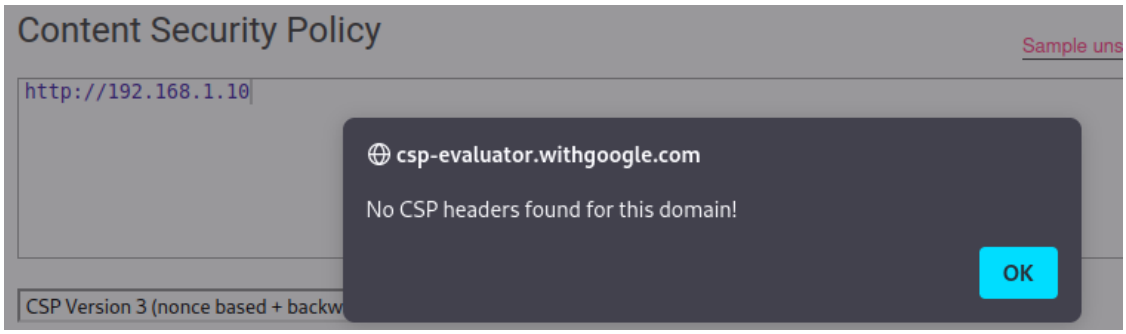


Fig 4.6 – Web application being tested for content security policy.

The evaluator indicates that the web application does not have any CSP headers within itself.

5 IDENTITY MANAGEMENT TESTING

5.1 TEST ROLE DEFINITIONS

The web application makes use of two types of users with differing levels of hierarchy. The first role would be the customer where anyone can freely make an account and login with it:

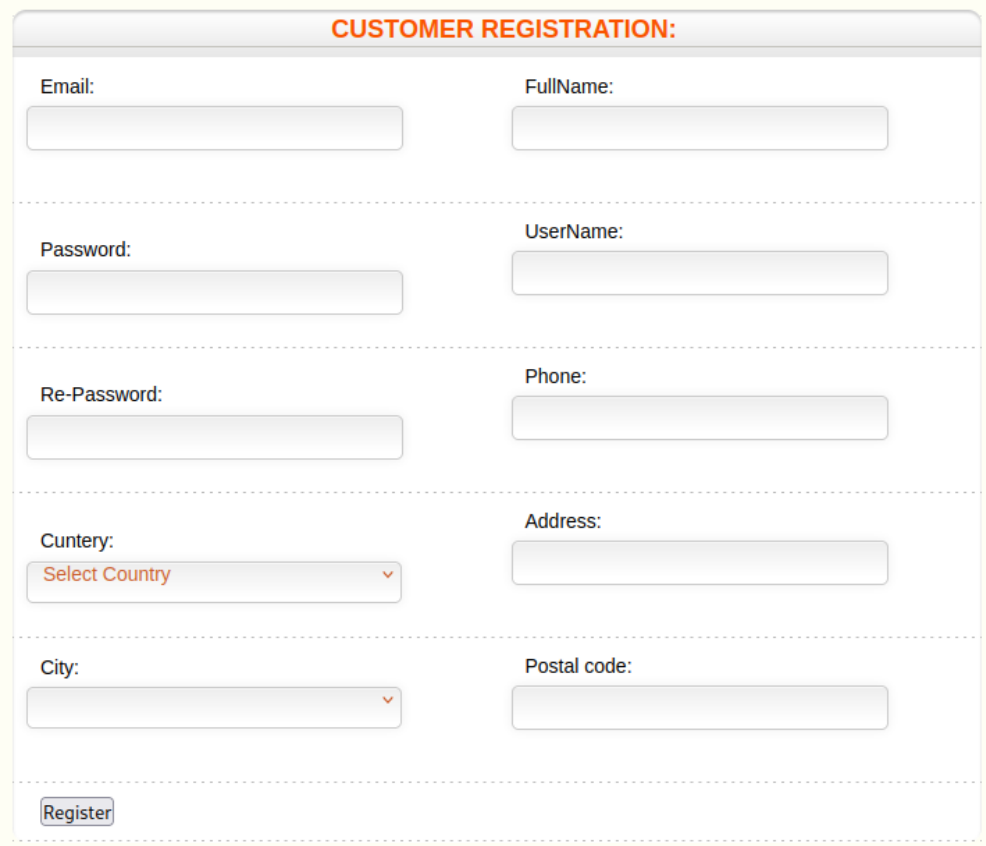


Fig 5.1– Registration form for customer role.

Meanwhile the employee/admin account registration form does not exist within the scope for an unauthenticated user. The login panel is accessible, however.

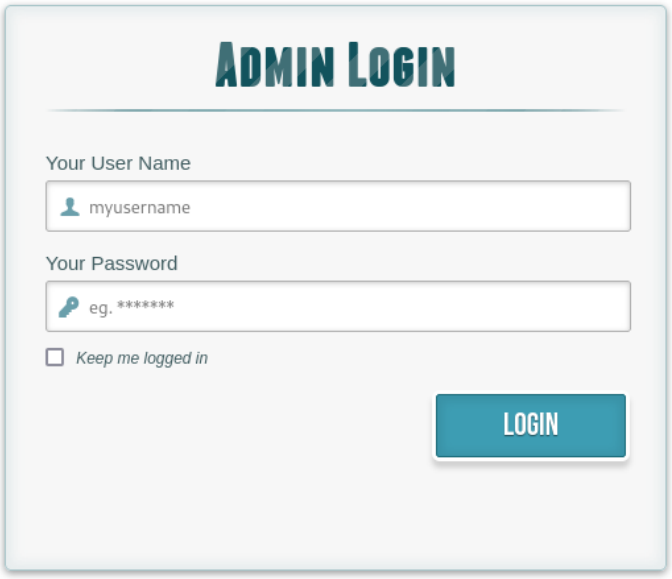


Fig 5.2 – Administrator login panel.

5.2 TEST USER REGISTRATION PROCESS

Regarding the customer account, the registration process is relatively simple.

- Anyone can register an account; no proof of identity is required.
- Registered accounts are not required to do any verification such as email verification to prove they are the owner of the email being used.
- User accounts can use the same email/phone number to sign up with another account.
- Character limit is 25 characters.
- Some of the fields are sanitised.

5.3 TEST ACCOUNT PROVISIONING PROCESS

As stated prior, any individual can create a customer account as many times as they like. Once created they have the ability to edit any of their information including changing their password.

Once administrator permissions have been achieved, it is possible to access the tables for the customer and employee accounts. Within these tables, the credentials and information are visible, including the passwords which are stored in plaintext. Administrators are able to change the usernames and passwords of other administrators too, requiring no additional authentication.

Employee Name	User Name	Password
no longer mr admin	notadmin	notpassword

Fig 5.3 – Administrators employee name, username and password being changed.

5.4 Testing for Account Enumeration and Guessable User Account

During the logging in process, once credentials (valid or not) have been inputted, the user is taken to the web page /userValidate.php. An incorrect password shows nothing, the website will just appear to fail to load, however an incorrect username creates a popup.

Fig 5.4 – Incorrect username popup from /userValidate.php.

Using this information, it is possible for an attacker to work out when a username is incorrect or not.

6 AUTHENTICATION TESTING

6.1 TESTING FOR DEFAULT CREDENTIALS

Default credentials are commonly in place for misconfigured networks and web applications. Common default credentials include common bad passwords such as ‘password’ or ‘admin’ and often are the same as the associated account username. Through the use of directory enumeration using dirbuster, a report was found leaking the usernames of administrators including one named ‘testadmin’. Attempting to log in with the credential’s username ‘testadmin’ and password ‘testadmin’ was successful leading to the administrator control panel.

6.2 TESTING FOR WEAK LOCK OUT MECHANISM

Web applications often feature a lockout system to prevent an attacker from utilising brute force password cracking techniques. In order to test the web applications lockout system, an account was created and then logging in with the correct password was performed 15 times with no lockout occurring.

6.3 TESTING FOR BYPASSING AUTHENTICATION SCHEMA

Authentication is a crucial aspect of a web application as it prevents unauthorised users from causing damage, either to other web users or to the entire website. There are numerous ways to bypass the authentication methods on a website.

Direct page request refers to the process of inputting the specific page wanting to access directly within the search engine, not requiring access to a prior page. The dirbuster report generated a list of accessible web pages which could be accessed through the search engine, including some within the /admin sub directory. These were automatically documents containing critical information and the customers and employees, see 4.3.

Parameter modification utilises the method of changing parts of the URL to trick the web application into thinking you are authenticated. Due to the web application not using the URL like this, it was not vulnerable to the exploit.

SQL injection is the process of inputting SQL queries into a HTML form to bypass authentication or leak information. Due to the importance of the attack, SQL injection will be elaborated further later in the report, regarding how it operates. The login form for customers is vulnerable to SQL injection. By inputting the username of an account that already exists and inputting “' OR 1=1;-- “ into the password box it is possible to bypass the authentication and log in with that user. Note that this did not work with the administrator’s login panel, only with the customers.

6.4 Testing for Weak Password Policy and Testing for Weak Security Question Answer

While web applications rarely create a password for the user, often they should guide the user with a password policy to create strong passwords to avoid them being a victim of brute forcing and password guessing. A good password policy would encourage customers to within their password include:

- Good length, ideally using more than one different word.
- Use of capital letters, numbers, and symbols.
- Not using common password words or personal details.

In order to test the web applications password policy, the worst possible password was used including a username comprised of numbers incrementing to test the upper limits of character allowed.

Fig 2.17 – Username and password found in the customer table for the account created the purpose of testing the password policy.

The creation of the account was successful with the application allowing it despite the terrible password. To confirm the account was successfully created, the customer table was accessed, and the new account was found. The character limit for the fields was also identified to be 20. Additionally, there are no security questions, so testing was not required. The user can change their password once logged in, however are prompted for their old password.

7 AUTHORIZATION TESTING

7.1 TESTING FOR PRIVILEGE ESCALATION

Often within a website there are levels of hierarchy which are used to prevent untrusted users from damaging the web application. Attackers often attempt vertical escalation where the goal is to gain privilege of the next level up. Horizontal escalation also exists however it is the method gaining access to accounts on the same level of power as the current account. A method of identifying what level of privilege a user had can be achieved using burpsuite web proxy to capture a HTTP POST packet and identify any user groups or variables potentially relating to the user's current privilege.

Fig 7.1 – HTTP POST packet from the web page /userValidate.php

From the packet, there are no group variable at all controlling the user's privilege making privilege escalation using this method most likely impossible. It could be assumed that the cookie or SecretCookie are responsible for controlling the users' privileges.

7.2 TESTING FOR INSECURE DIRECT OBJECT REFERENCES

Using the search engine, it is possible to treat it similar to a command line by inputting certain parameters to access certain parts of the web application that are normally unobtainable. Within the administrator panel, it is possible to view, update, add, and delete entries within the tables. Each entry with the table has a unique ID which can be used from the search engine to identify it.

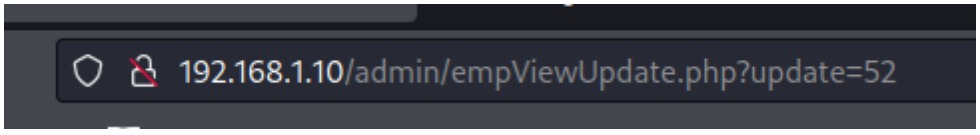


Fig 7.2 – Using the search engine to find the entry with the ID of 52.

Typically, all the entries are accessible already however if there are ever any hidden entries, they could be identified using this method.

8 SESSION MANAGEMENT TESTING

8.1 TESTING FOR SESSION MANAGEMENT SCHEMA

Cookies are used within web applications to maintain the user's state including their credentials. Cookies are intended to be unique to an individual and encoded enough to not give away their data. It is possible to access the user's own cookies within the web browser via the cookie jar. Upon enumerating the website, it was discovered that 2 cookies exist, accessing these cookies can be achieved using burpsuite or OWASP ZAP. These 2 cookies were 'Cookie' and 'SecretCookie'. To understand the purpose and how the cookies are encoded, two separate accounts with slightly different details were created with their cookies saved. Both accounts had the same password, being 'password', but different usernames.

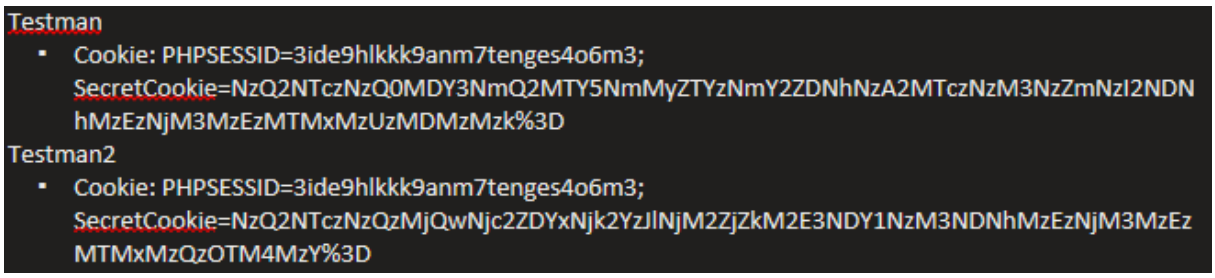


Fig 8.1 – Cookies for 2 separate accounts created on the web application.

Firstly, 'Cookie' between the two accounts appears to be identical. This would indicate that the first cookie does not have anything to do with the credentials and authentication of the account.

The 'SecretCookie' on the other hand varies greatly between the two accounts. Often cookies are encoded using various different methods and usually it is possible to detect which encoding is being used by identifying certain aspects of the encoded string. For example, the '%' at the end of the cookie suggests that it is encoded using URL encoding. Running it through a URL decoder results in the following string:

```
NzQ2NTczNzQ0MDY3NmQ2MTY5NmMyZTYzNmY2ZDNhNzA2MTczNzM3NzZmNzI2NDNhMzEzNjM3MzEzMT
MxMzUzMDMzMzk=
```

Fig 8.2 – 'Testman' SecretCookie ran through a URL decoder.

The '=' indicates that the current string is encoded using base64, which can be decoded using an online decoder again:

```
7465737440676d61696c2e636f6d3a70617373776f72643a31363731313135303339
```

Fig 8.3 – Previous string decoded using base64.

Judging from the only characters being present only including ones found within a hexadecimal range, the string was run through a hexadecimal decoder.

Fig 8.4 – The email address and password of the user can be found within the SecretCookie.

The email address and password used for the account can be found within the SecretCookie. This makes any exploit involving stealing the user's cookie more critical due to personal details being easily leaked.

8.2 Testing for Exposed Session Variables

Attackers using session tokens can impersonate a user and authenticate themselves on the web application. To do this, the session tokens must be stolen using malicious methods such as eavesdropping.

As enumerated prior, the web application makes use of HTTP, meaning that any packets intercepted via eavesdropping will remain unencrypted resulting in Session IDs and cookies being vulnerable and potentially stolen.

Using the Burpsuite web proxy, it is possible to intercept and capture a HTTP POST packet including the user's cookies.

```
11 Referer: http://192.168.1.10/login.php
12 Cookie: PHPSESSID=3ide9hlkkk9anm7tenges4o6m3; SecretCookie=NzQ2NTczNzQ0MDY3NmQ2MTY5NmM
13 Upgrade-Insecure-Requests: 1
14
15 magaca=test%40gmail.com&furaha=password&submit=+Login
```

Fig 8.5 – Packet intercepted from /userValidate.php when attempting to log in.

Not only are the cookies available within the packet, the email and password are clearly shown. Poorly configured web pages can use the information on the bottom line to login using the search bar.

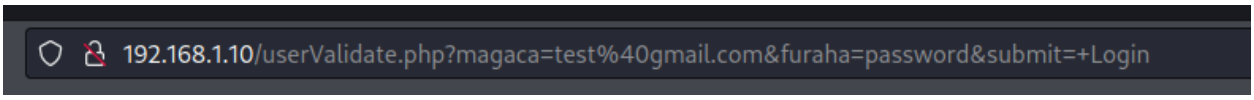


Fig 8.6 – Testing vulnerability for logging in using the search bar on HTTP POST packets.

The test was unsuccessful meaning the application was not vulnerable to this specific exploit.

8.3 Testing for Logout Functionality

The logout functionality within a website is crucial to protecting a user's session and cookies. When a user logs out of a website, the session cookie should immediately be terminated. Within the web application, when a user has successfully logged in, a logout button should be present on all pages.

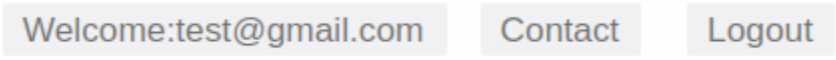


Fig 8.7 – Log out button found on the web application next to the user's email address.

The logout button for customers was located at the top right of the website, however depending on certain pages, such as when finalising purchases and payment, the logout button is not present. Additionally, the user's cookie and SecretCookie persists even when logged out, meaning an attacker could steal the cookie even after the user has seemingly logged out.

8.4 TESTING FOR SESSION HIJACKING

As discussed before, cyber criminals can hijack a user's cookies to authenticate themselves without needing their credentials. To test whether an attacker can utilise this exploit, the an account was logged in before accessing their cookies within the cookie jar (where cookies are stored) of a web browser.

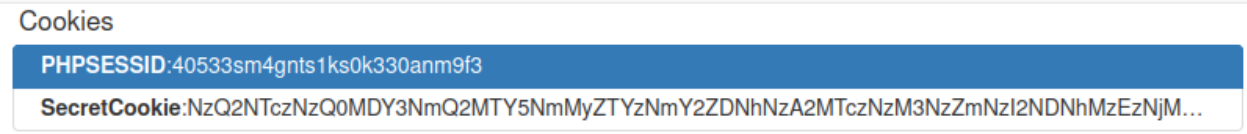


Fig 8.8 – Cookies accessed within cookier jar.

The cookies accessed were then copied and saved to be used later and afterwards cleared to remove any evidence the user was logged in. A different account was logged in, immediately logged out and removed from the cookie jar out to completely refresh the system. Finally, the prior saved cookies were added back to the cookie jar. The account was then able to successfully log in.

9 INPUT VALIDATION TESTING

9.1 TESTING FOR REFLECTED CROSS SITE SCRIPTING – EVALUATE

Reflected Cross site scripting attacks is when malicious code that can be executed by the browser into a HTTP response. The attack will only affect users who open links specifically for the exploit. The attacking code is often written in Javascript meaning the input vectors often are sanitised. A method for testing which input vectors allow for XSS include inputting a very basic javascript line of code into the chosen input vector. The account creation page, change details page, customer login form and admin login form were all assumed to be potential input vectors.

Firstly, the change details page was tested for potential reflected XSS due to being able to update regularly allowing for many tests quickly. The first issue was that when testing the code '<script>alert(123)</script>', it would be cut off due to the character limit being 25. The work around this was to reduce the numbers within the alert to just 1.

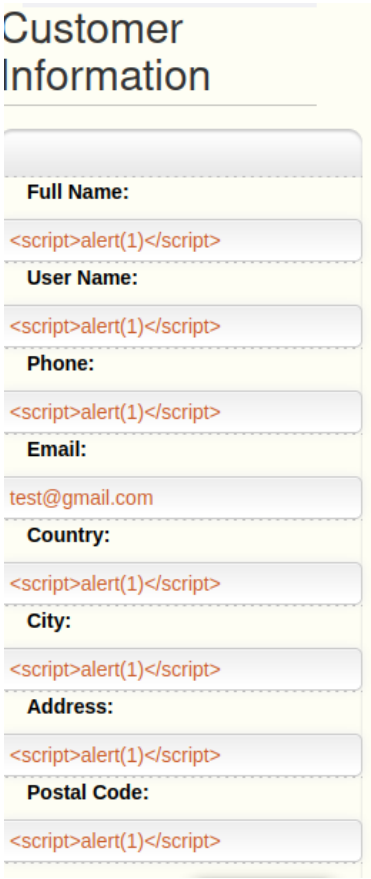


Fig 9.1 – Customer information on /profile.php where input vectors for XSS are being tested.

All the fields with the exception of the email allowed the line of code to go through. Initially it appeared that the code did not work as there was no alert popup however this is not the case when accessing the admin subdirectory. This indicates that this is instead a stored XSS attack.

9.2 TESTING FOR STORED CROSS SITE SCRIPTING

The more deadly variety of Cross Site Scripting attack would be the stored ones. Stored XSS happens when the injected code is stored within the application, meaning that it could run the malicious code inputted by an attacker as the website.

As previously discovered, it is possible to create a stored XSS attack using customer profile page when updating the fields for the user. When accessing the customer table within the administrator panel, an alert is shown:

Fig 9.2 – Alert as a result of XSS coming from a customer profile onto an administrator table.

Due to the character limit imposed by the forms when updating details, creating damaging exploits would be tricky using this input vector. This does confirm that XSS vulnerabilities do exist within the website and that the next step should be attempting to find an input vector with a larger upper character limit.

Using this information, the administrator panel was searched for input vectors. The main purpose of the administrator panel and subdirectory was to manage the tables of the website. As previously mentioned, there were tables for customers, employees and products but also warehouses and categories. The category table featured a field called 'Description' which could be used for larger XSS attacks due to its larger field length. Using the insert button, it is possible to add a new entry to the table:

Fig 9.3 – Inserting an entry onto the categories table to create a stored XSS attack.

The payload for the attack is found within the description field with the attack displaying the user's cookies when they access the categories web page.

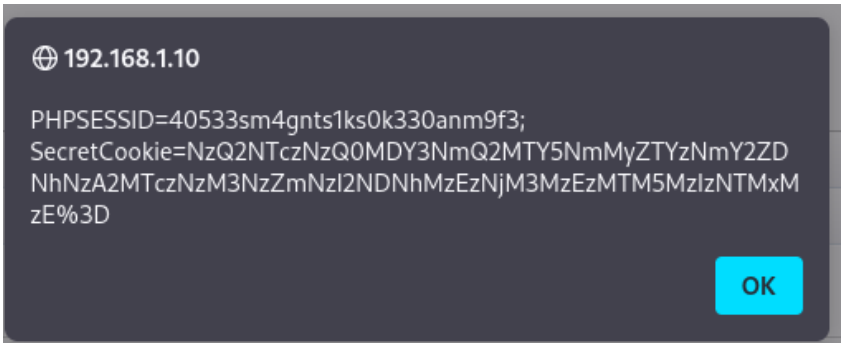


Fig 9.4 – User's cookies displayed within the alert via stored XSS.

9.3 TESTING FOR SQL INJECTION

SQL injection involves the process of injecting lines of code which resemble SQL queries into the web application to be incorrectly processed. These attacks can lead to data leakage and unauthorised access.

When testing authentication methods, it was found that the customer login page was vulnerable SQL injection. The exploit involved using an email address of a user and within the password form, typing in the code "' OR 1=1;-- ". This would be considered a severe SQL injection vulnerability as with the customer report leaked through directory traversal, the emails of users were vulnerable, leading to the ability to log in with any user. The specific SQL query works due to the "OR 1=1" telling the database to login if the input matches the password or if 1=1, which is always going to be true.

Through prior enumeration, it is known that the web application makes use of MySQL. Using this information, SQLMap, a tool found on Kali can be used to find other suitable input vectors for SQL injection using automated tools. A scan was firstly performed starting at the index page however this netted 0 results. The scan was then performed for the login page using the following command and configuration:

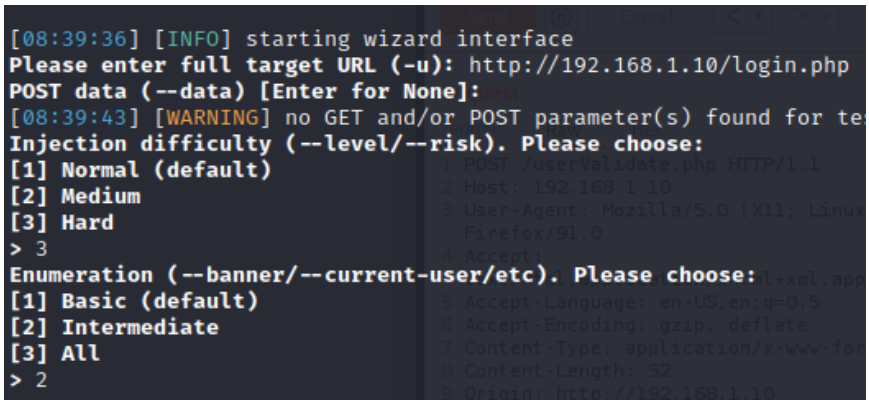


Fig 9.5 – SQL injection performed for the web applications login page.

The scan found a SQL injection point for the UserValidate.php web page, which was used to authenticate users and redirects unauthenticated users. SQLMap then injected a Boolean-based blind and time-based blind injection.

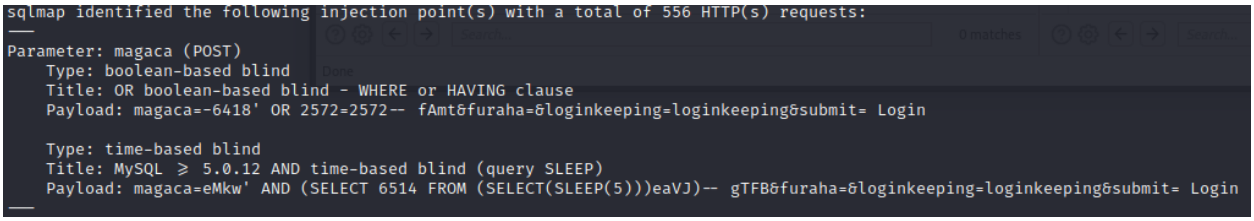


Fig 9.6 – SQLMap injecting into UserValidate.php.

A Boolean-based exploit relies on sending SQL queries in which the application responds with either a TRUE or FALSE which allows an attacker to enumerate an entire database character by character, resulting in massive information leak. The time-based exploit operates similarly however relies on the difference in response times to determine if the result of the query is TRUE or FALSE. The exploit goes further and identifies critical information about the database as a whole:

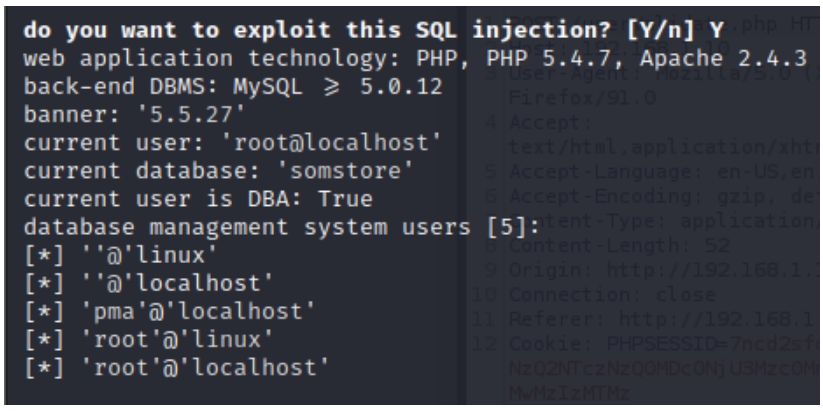


Fig 9.7 – Information regarding the database and it's managers.

Including the leaked managers, table names for the database Rickstores uses were leaked:

Fig 9.8 – Entire database used within Rickstores web application.

Additionally, other databases (that did not appear to be related to the web application) were leaked as well.

9.4 TESTING FOR COMMAND INJECTION

One of the main goals for an attacker is to be able to run operating system commands on the victim's machine. These commands can be used to gain the passwords and conduct further exploits on the victim's machine. Usually, the type of commands that can be executed depends on the file type of the web page. Majority of the web pages within the website appear to be PHP which did not appear to be vulnerable.

10 TESTING FOR ERROR HANDLING

10.1 TESTING FOR IMPROPER ERROR HANDLING

Error messages for applications ae generally used by web developers to understand what caused the issues. Commonly, web developers assume that users will use their website correctly and not cause any intentional issues resulting in error messages being incorrectly configured.

A common error message can be from HTTP error response codes. Accessing a web page that does not exist within the web application results in an error 404 response.

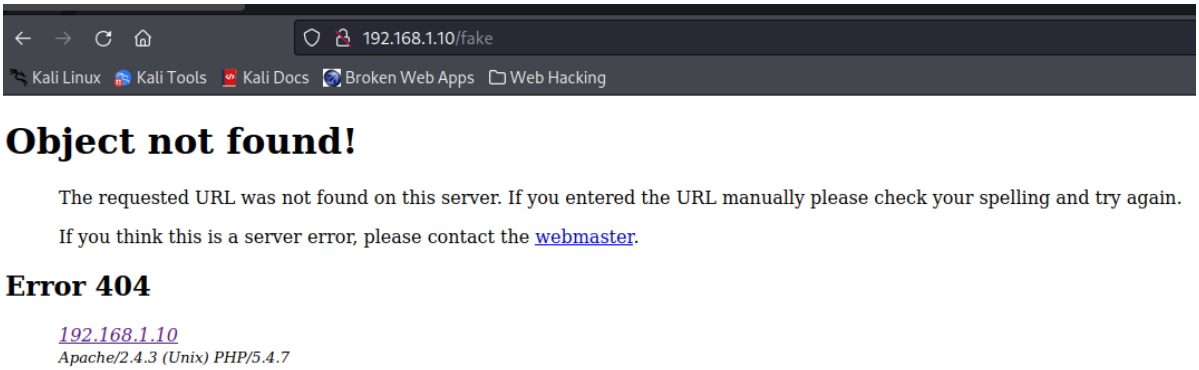


Fig 10.1 – Browsing to a web page that does not exist, such as 192.168.1.10/fake results in an error 404.

As seen from the error message, there is some information leakage including information about the web server.

11 TESTING FOR WEAK CRYPTOGRAPHY

11.1 TESTING FOR WEAK TRANSPORT LAYER SECURITY

Web applications when communicating information between the client and the web server need to be encrypted. The NMAP scans performed prior indicated that the web application was running HTTP, which is an unencrypted service resulting in no Transport Layer Security (TLS).

11.2 TESTING FOR SENSITIVE INFORMATION SENT VIA UNENCRYPTED CHANNELS

Due to the lack the of encryption present in the web application since it uses HTTP instead of HTTPS, it is possible for sensitive information to eavesdropped on. Sensitive information can include credentials and cookies to Personal Identifying Information (PII) such as bank account numbers and social security numbers. From prior enumeration and access to the administrators' database including all the personal details stored, no PII could be found. However, it is also known that credentials are kept in plaintext within the website, including administrator credentials. It is possible to access the administrator website with curl requiring no authentication:

```
┌──(kali㉿kali)-[~]
└─$ curl -kis http://192.168.1.10/admin/
HTTP/1.1 302 Found
Date: Thu, 22 Dec 2022 15:15:21 GMT
Server: Apache/2.4.3 (Unix) PHP/5.4.7
X-Powered-By: PHP/5.4.7
Set-Cookie: PHPSESSID=oehp84r795b0lacegsprpc6c72; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
Pragma: no-cache
Location: ../index.php?Not logged in
Transfer-Encoding: chunked
Content-Type: text/html

You Failed !!
<!doctype html>
<html lang="en">

<head>
        <meta charset="utf-8" />
        <title> RickStore I Admin </title>
<link href="css/bootstrap.min.css" rel="stylesheet" />
      <link href="css/bootstrap.min.css" rel="stylesheet" />
        <link rel="stylesheet" href="css/layout.css" type="text/css" media="screen" />
<link rel="shortcut icon" href="images/favicon.png" />
         <link rel="stylesheet" href="css/chatStyle.css" type="text/css" media="screen" />
```

Fig 11.1 – HTTP packet contents of 192.168.1.10/admin using.

The administrator subdirectory includes all the tables used for authentication and tracking of orders and products leading to information being leaked if visiting the correct table page.

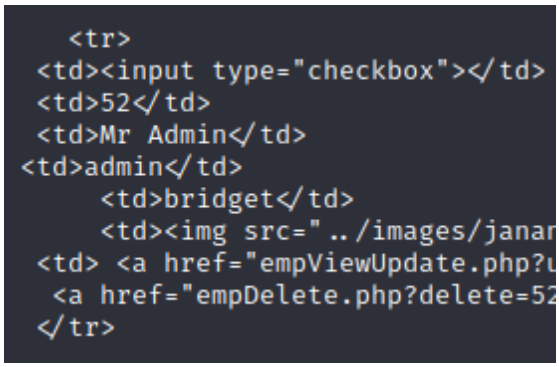


Fig 11.2 – Administrator login details found using curl on 192.168.1.10/admin/Employees.php.

Within the HTML code, the administrator credentials, Mr Admin and bridget can be found, meaning any unauthenticated user could find these details and log in with them. Additionally, only 1 cookie is present when using curl, the cookie found out to have nothing to do with the authentication of the user.

12 DISCUSSION

12.1 OVERALL DISCUSSION

The penetration test for Rickstore groups was overall a success with numerous vulnerabilities being identified for various aspects of the web application. Following the OWASP methodology was successful due to the comprehensive testing it ensured with useful descriptions of what the vulnerability was, how to perform the exploit and useful tools to use during the penetration test. The specific aims were also achieved with varying levels of success. For example, identifying SQL injection points and obtaining administrator credentials was very successful however there was less success with authorisation errors due to time and ability constraints.

12.1.1 Information gathering

The information gathering phase of the penetration test went smoothly leading to many useful starting points crucial to performing exploits. The information found through the enumeration of the robots.txt web page may have had not much information useful to a penetration tester (i.e., no credentials) however it did include numerous spreadsheets worth of information regarding customers and how the business was performing which would be extremely valuable to a malicious hacker, especially considering this would be considered low hanging fruit requiring very little technical prowess to acquire. Identifying the webserver was useful for identifying surrounding exploits while also being relatively easy to perform due to the information being leaked in a Nikto scan and in HTTP packets. Another scanning tool used was NMAP which identified numerous services running, including MySQL which greatly helped in performing SQL injection exploits later. Using the spidering feature in OWASP ZAP was only reasonably helpful due to a lot of false positives being identified in the output and other tools such a dirbuster getting the same results and identifying usually inaccessible web pages. Interestingly, no web application frameworks could be identified using the recommended tools, this could be either due to poor testing or lack of any present in the web application. Finally, the mapping of the architecture was greatly beneficial due to creating an easy-to-understand summary of the information acquired.

12.1.2 Configuration and deployment management testing

Commonly, attackers will rely on errors during the configuration of a website in order to deploy exploits and attacks. Some of these commonly known misconfigurations were found within the Rickstores web application. Various automated scanners for detecting vulnerabilities due to misconfigurations exist, including OWASP ZAP scanner which identified numerous misconfigurations such as the authentication page being vulnerable to SQL injection and only using HTTP. Both of these lead to critical exploits later within the penetration test. Default credentials of the website were not present found through testing however eventually gaining access to the database for employees, an account resembling the structure a default credential follows (username the same as the password) were eventually found however it was not quite guessable. Additionally, due to not identifying any framework it was not possible to research any credentials surrounding that. Using dirbuster was arguably the most useful tool used during the penetration test due to it identifying key web pages within the network containing sensitive information which would enable an attacker to gain administrator authentication. Dirbuster identified the administrator subdirectory however majority of its contents required administrator credentials to access.

Curiously, 2 documents did not require this and could be accessed via the browser easily. Reasoning as to why these did not require authentication could be because these web pages were documents that were generated and updated each time the user clicked on the link to them, meaning the authentication code could have easily been left out.

12.1.3 Identity management testing

The web application had 2 separate roles for accounts, employees and customers with employees unsurprisingly having elevated privileges. The process for creating a customer account can be done by anyone compared to employee accounts which require administrator authentication. During the user registration process, invalid or made-up details can be used, which could result in bots mass creating accounts with email addresses they do not own. A potential devastating flaw if an attacker successfully compromises a website is the administrator's ability to change the passwords of other administrators, requiring no other outside authentication, resulting in potentially locking out the entire website.

12.1.4 Authentication testing

The authentication process of the website revolved around the webpage /userValidate.php which was found to be vulnerable to SQL injection. The SQL injection attack results in a password not being required to log in, being incredibly severe. Interestingly, only the customer login page is vulnerable to this attack, suggesting that more input checks and safety measures were present for the administrator login. The web application lacks surrounding security regarding keeping its customers' accounts safe such as encouragement for strong passwords and a lockout system for absurd amounts of attempts.

12.1.5 Authorisation testing

Authorisation testing did not net many results, mainly due to many of the tests not applying to the website and were therefore skipped. For example, no groupID could be found to relate to the current hierarchy of an account. Examples of where there was an exploit, such as insecure direct object references were often pointless due to all the information being present and available to an attacker at that point.

12.1.6 Session Management testing

The session tokens used within the website stem from 2 cookies, however it was found that only 1, named 'SecretCookie', This cookie was able to be decoded using various common decoding methods and due to the unencrypted HTTP service in place, if an attacker was to eavesdrop, then their credentials would be comprised. Session hijacking would also be a serious vulnerability if cookies were stolen by an attacker using various methods such as clickjacking.

12.1.7 Input Validation testing

During the input validation process, the website was found to be vulnerable to numerous XSS and SQL injection attacks. To begin with the XSS attacks, at first various input vectors were identified, most notably the account creation page and eventually the change details page. An issue arose when the character limit was found to just be 25 resulting in very limited possible commands which could be added to the input vector. It was discovered that using these input vectors lead to a stored XSS attack, instead of a reflected which is far worse. The attack from the customers profile, ends up causing damage for the administrator as the script is accessed when opening the customers table. A method of causing damage to users accessing the webstore was to insert an XSS payload into the description field for categories, allowing for much larger attacks such as stealing cookies. The SQL injection attacks include bypassing customer password authentication, which when including the customer email address leaks found in

/CustomerReport.php, can lead to any account being compromised. Additionally using SQLMap resulted in a very large data leak due to the Boolean and time-based exploits identified. Once the entire database had been found, attempts to manually find the tables within the admin subdirectory using directory traversal was attempted however was unsuccessful.

12.1.8 Testing for error handling

The error handling was very poor, going as far as to leak the type of error and the web server type and service. Interestingly, when also testing for directory error handling, the error was not the intended one to be expected, as the /js directory was completely accessible with 0 restrictions.

12.1.9 Testing for weak cryptography

Due to the websites severe lack of encryption, TLS is not present which is incredibly insecure. Additionally, while testing for accessing cookies unencrypted cookies using curl, it was discovered that it was possible to access the administrator subdirectory and all the tables within it. Additionally, all the data found within the tables, most notably passwords, were stored in plaintext and should have been hashed to begin with.

12.2 COUNTERMEASURES

12.2.1 Information gathering

Counteracting information gathering is a tricky process which involves removing as much information which is accessible to outside attackers. To prevent information leakage from robots.txt, using a directory to hold the files inside and mark the directory in robots.txt, then using a redirect page if an attacker enters the directory. Regarding scanners, using a firewall to block information about the services running can be beneficial, NMAP creators however do not suggest utilising defensive software to combat port scanners since often it can lead to more vulnerabilities.

12.2.2 Configuration and deployment management testing

Counteracting dirbuster is difficult due to the method of brute forcing common website names, which are named for user-friendliness. However, ensuring that all files which are to be protected by administrator authentication are definitely protected will prevent any unauthorised access due to misconfigurations, unlike the generated employee reports. Should the web application utilise HTTPS, strict transport security should be enabled, and Content Security Policy headers should be added to help prevent XSS where possible.

12.2.3 Identity management testing

The website should investigate users needing to verify their email address to fully enable their account as well as correctly validating phone numbers and location. Email address verification is as simple as sending a unique link to the submitted email address and once clicked, their account is valid. Depending on how much the administrators care about phone numbers and addresses, a method of validating the two together would be to use the first numbers within a phone number to identify the region it belongs to and compare it to the country the user submits. For example, if a user submitted a phone number beginning with +44 and said they lived in Austria, the website could reject this account being made.

12.2.4 Authentication testing

The 'testadmin' account found should use improved password security by using a different password, one not matching its username and not be related to the fact that its used for testing administrator roles, in order to make it less guessable. The website should deploy some sort of lockout system if too many failed attempts are made. Additionally, if the lockout is activated, it may be worth alerting the administrators and user of said account that a potential brute force attack is taking place. The website should greatly encourage customers to use stronger passwords as there are 0 recommendations in place in order to prevent weak passwords. The same should also be said for employees, if not encouraged more. Passwords should contain:

- Numerous words, ideally not relating to the user to avoid social engineering attacks.
- Upper case letters, numbers, and symbols.
- Ideally different than other passwords made by the same user in case there's ever a leak involving the same passwords.

12.2.5 Input validation testing

With the XSS vulnerabilities present, the website should use more sanitising for the input vectors. GoMakeThings recommend using DOMPurify due to it being highly configurable and can use a list to allow specific characters if necessary. A simpler countermeasure to use before setting up DOMPurify would be the function htmlspecialcharacters() which makes constructing commands for XSS more difficult for attackers by changing certain characters such as >. As stated before, Content Security Policy headers should also be used to prevent XSS attacks. The customer login page was found to be severely vulnerable to SQL injection, to protect against this, prepared statements should be used within PHP to separate the input data from the code.

12.2.6 Testing for error handling

To improve how errors are handled within the web page, web administrators should investigate having a web page specifically for web page errors. This can be achieved by having users redirected to the error web page. Specifying the type of error is optional as it can improve user experience however it does leak information to hackers.

12.2.7 Recommendation for encryption and HTTPS

Due to the unencrypted nature of the website, the sessions tokens/cookies of the user can be compromised meaning utilising TLS should be mandatory. For TLS to operate, HTTPS must be in place. The web administrators should greatly consider switching to the HTTPS protocol instead of HTTP. The unencrypted packets allow for numerous eavesdropping situations to arise. Additionally, passwords and other sensitive information should absolutely not be saved in plaintext. Passwords should be hashed using an appropriate hashing function such as SHA256.

12.3 FUTURE WORK

While following the OWASP methodology, certain tests could not be performed mainly due to 3 reasons:

- Test appeared relatively pointless or impossible (mainly due to software not present on the web application) so the test was skipped to save time.
- Test was too difficult to perform on this web application.

- Tests towards the end were skipped due to running low on time.

Numerous exploits required hosting a website to perform such as some input validation tests. If given more time and resources, then these tests would have been performed. More time dedicated to SQL injection would have been preferred, especially after SQLMap discovered the massive database leak. Investigating clickjacking capabilities would also have been tested given more time.

Once Rickstore has correctly setup HTTPS and encryption, a further penetration test would be recommended to ensure that the new software and services had been setup correctly.

REFERENCES

Baig, A. (2021, July 9). *What is session hijacking and how do you prevent it?* Retrieved from GlobalSign: https://www.globalsign.com/en/blog/session-hijacking-and-how-to-prevent-it

Chapter 11. Defenses against NMAP. (n.d.). Retrieved from NMAP: https://nmap.org/book/defenses.html

chinmay_bhide. (2021, December 17). *How to prevent xss with html php*. Retrieved from geeksforgeeks: https://www.geeksforgeeks.org/how-to-prevent-xss-with-html-php/

Cost of a data breach 2022. (2022). Retrieved from IBM: https://www.ibm.com/uk-en/reports/data-breach#

Foundation, O. (2020, April). *Web Application Security Testing*. Retrieved from OWASP: https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/

Muscat, I. (2019, March 27). *How to prevent SQL injection vulnerabilities in PHP applications*. Retrieved from Acunetix: https://www.acunetix.com/blog/articles/prevent-sql-injection-vulnerabilities-in-php-applications/

Retail e-commerse sales worldwide from 2014 to 2026. (2022, July). Retrieved from Statista: https://www.statista.com/statistics/379046/worldwide-retail-e-commerce-sales/

Types of SQL injection (SQLi). (n.d.). Retrieved from Acunetix: https://www.acunetix.com/websitesecurity/sql-injection2/

Watts, S. (2019, January 31). *Robots txt security Risks*. Retrieved from Search Engine Journal: https://www.searchenginejournal.com/robots-txt-security-risks/289719/#close

(Watts, 2019)

(chinmay_bhide, 2021)

(Muscat, 2019)

(Baig, 2021)

(Types of SQL injection (SQLi), n.d.)

(Retail e-commerse sales worldwide from 2014 to 2026, 2022)

(Cost of a data breach 2022, 2022)

(Chapter 11. Defenses against NMAP, n.d.)

(Foundation, 2020)

APPENDICES PART 1

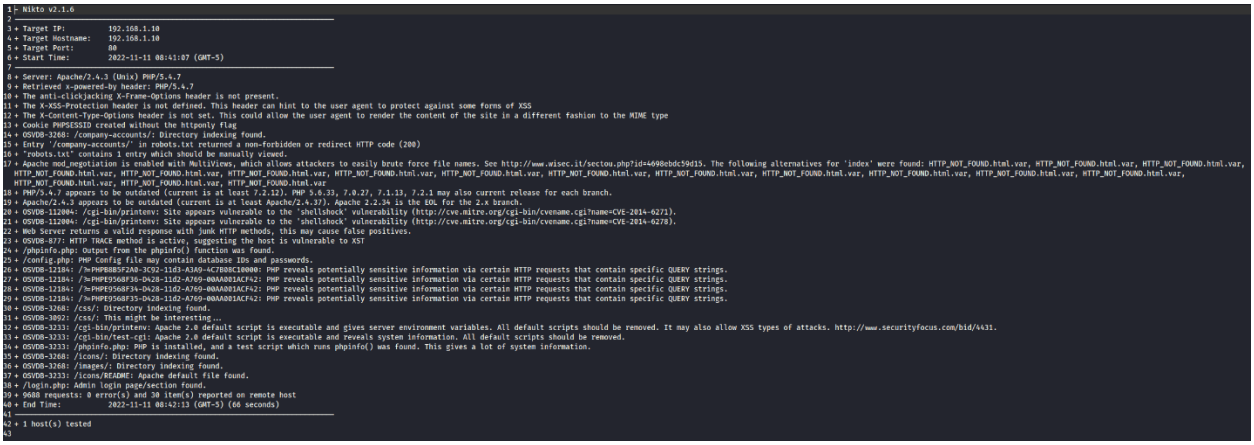
APPENDIX A OWASP ZAP AUTOMATED SCAN

```
Processed,Method,URI,Flags
true,GET,http://192.168.1.10,Seed
true,GET,http://192.168.1.10/robots.txt,Seed
true,GET,http://192.168.1.10/sitemap.xml,Seed
true,GET,http://192.168.1.10/company-accounts,
true,GET,http://192.168.1.10/,
true,GET,http://192.168.1.10/contact.php,
true,GET,http://192.168.1.10/company-accounts/,
true,GET,http://192.168.1.10/login.php,
true,GET,http://192.168.1.10/index.php,
true,GET,http://192.168.1.10/products.php,
true,GET,http://192.168.1.10/warehouse_1.php,
true,GET,http://192.168.1.10/warehouse_2.php,
true,GET,http://192.168.1.10/about.php,
true,GET,http://192.168.1.10/customer.php,
true,GET,http://192.168.1.10/affix.php?type=terms.php,
true,GET,http://192.168.1.10/images/favicon.png,
true,GET,http://192.168.1.10/css/style.css?version=17,
true,GET,http://192.168.1.10/css/proStyle.css,
true,GET,http://192.168.1.10/css/userlogin.css,
true,GET,http://192.168.1.10/css/cart.css?version=1,
true,GET,http://192.168.1.10/css/chatStyle.css,
true,GET,http://192.168.1.10/css/audioplayer.css,
true,GET,http://192.168.1.10/js/jquery-1.6.2.min.js,
true,GET,http://192.168.1.10/js/cufon-yui.js,
true,GET,http://192.168.1.10/js/Myriad_Pro_700.font.js,
true,GET,http://192.168.1.10/js/jquery.jcarousel.min.js,
true,GET,http://192.168.1.10/js/functions.js,
true,GET,http://192.168.1.10/js/main.js,
true,GET,http://192.168.1.10/images/logo.png,
true,GET,http://192.168.1.10/images/a.jpg,
true,GET,http://192.168.1.10/images/b.jpg,
true,GET,http://192.168.1.10/images/s1.jpg,
true,GET,http://192.168.1.10/images/cart.jpg,
true,GET,http://192.168.1.10/images/s2.jpg,
true,GET,http://192.168.1.10/images/s3.jpg,
true,GET,http://192.168.1.10/images/s4.jpg,
true,GET,http://192.168.1.10/images/s5.jpg,
true,GET,http://192.168.1.10/images/s6.jpg,
true,GET,http://192.168.1.10/images/social-icon1.png,
true,GET,http://192.168.1.10/images/social-icon2.png,
true,GET,http://192.168.1.10/images/social-icon4.png,
true,GET,http://192.168.1.10/images/social-icon7.png,
true,GET,http://192.168.1.10/images/cart-img1.jpg,
true,GET,http://192.168.1.10/images/cart-img2.jpg,
true,GET,http://192.168.1.10/images/cart-img3.jpg,
true,POST,http://192.168.1.10/cart_update.php,
true,POST,http://192.168.1.10/cart_update.php,
true,POST,http://192.168.1.10/cart_update.php,
true,POST,http://192.168.1.10/cart_update.php,
true,POST,http://192.168.1.10/cart_update.php,
true,POST,http://192.168.1.10/cart_update.php,
true,GET,http://192.168.1.10/company-accounts/?C=N;O=D,
true,GET,http://192.168.1.10/Sign%20In.php,
true,GET,http://192.168.1.10/company-accounts/?C=M;O=A,
true,GET,http://192.168.1.10/company-accounts/?C=S;O=A,
true,GET,http://192.168.1.10/css/style.css,
true,GET,http://192.168.1.10/company-accounts/?C=D;O=A,
true,GET,http://192.168.1.10/css/cart.css,
true,GET,http://192.168.1.10/company-accounts/finances.zip,
true,GET,http://192.168.1.10/company-accounts/readme.txt,
true,GET,http://192.168.1.10/icons/blank.gif,
true,GET,http://192.168.1.10/icons/back.gif,
true,GET,http://192.168.1.10/icons/compressed.gif,
```

```
 64 true,GET,http://192.168.1.10/icons/compressed.gif,
 65 true,POST,http://192.168.1.10/feedback_process.php,
 66 true,GET,http://192.168.1.10/icons/text.gif,
 67 true,GET,http://192.168.1.10/css/demo.css,
 68 true,GET,http://192.168.1.10/css/AnimateLogo.css,
 69 true,GET,http://192.168.1.10/css/animate-custom.css,
 70 false,GET,http://www.css3create.com/Astuce-Empecher-le-scroll-avec-l-utilisation-de-target,Out of Scope
 71 true,POST,http://192.168.1.10/userValidate.php,
 72 true,POST,http://192.168.1.10/employeeValidate.php,
 73 true,GET,http://192.168.1.10/products.php?command&productid,
 74 true,POST,http://192.168.1.10/cart_update.php,
 75 true,POST,http://192.168.1.10/cart_update.php,
 76 true,POST,http://192.168.1.10/cart_update.php,
 77 true,POST,http://192.168.1.10/cart_update.php,
 78 true,POST,http://192.168.1.10/cart_update.php,
 79 true,POST,http://192.168.1.10/cart_update.php,
 80 true,POST,http://192.168.1.10/cart_update.php,
 81 true,POST,http://192.168.1.10/cart_update.php,
 82 true,POST,http://192.168.1.10/cart_update.php,
 83 true,POST,http://192.168.1.10/cart_update.php,
 84 true,POST,http://192.168.1.10/cart_update.php,
 85 true,POST,http://192.168.1.10/cart_update.php,
 86 true,GET,http://192.168.1.10/warehouse_1.php?command&productid,
 87 true,POST,http://192.168.1.10/cart_update.php,
 88 true,POST,http://192.168.1.10/cart_update.php,
 89 true,POST,http://192.168.1.10/cart_update.php,
 90 true,GET,http://192.168.1.10/warehouse_2.php?command&productid,
 91 true,POST,http://192.168.1.10/cart_update.php,
 92 true,POST,http://192.168.1.10/cart_update.php,
 93 true,POST,http://192.168.1.10/cart_update.php,
 94 true,GET,http://192.168.1.10/images/profile1.jpg,
 95 true,GET,http://192.168.1.10/images/profile2.jpg,
 96 true,GET,http://192.168.1.10/js/countries.js,
 97 true,POST,http://192.168.1.10/insertCustomer.php,
 98 false,GET,http://stackoverflow.com/questions/2610497/change-an-inputs-html5-placeholder-color-with-css,Out of Scope
 99 false,GET,http://jquery.com/,Out of Scope
100 false,GET,http://jquery.org/license,Out of Scope
101 false,GET,http://sizzlejs.com/,Out of Scope
102 false,GET,http://sorgalla.com/jcarousel/,Out of Scope
103 false,GET,http://sorgalla.com/,Out of Scope
104 false,GET,http://www.opensource.org/licenses/mit-license.php,Out of Scope
105 false,GET,http://www.opensource.org/licenses/gpl-license.php,Out of Scope
106 false,GET,http://billwscott.com/carousel/,Out of Scope
107 true,GET,http://192.168.1.10/company-accounts/?C=N;O=A,
108 true,GET,http://192.168.1.10/company-accounts/?C=M;O=D,
109 true,GET,http://192.168.1.10/company-accounts/?C=S;O=D,
110 true,GET,http://192.168.1.10/company-accounts/?C=D;O=D,
111 true,GET,http://192.168.1.10/thankyou.php?id=ZAP,
112 false,GET,http://daneden.me/animate/,Out of Scope
113 false,GET,http://daneden.me/animate,Out of Scope
114 false,GET,https://github.com/nickpettit/glide,Out of Scope
115 true,GET,http://192.168.1.10/cart_update.php?removep=1&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC9wcm9kdWN0cy5waHA/Y29tbWFuZCZwcm9kdWN0aWQ=,
116 true,GET,http://192.168.1.10/cart_update.php?removep=2&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC9wcm9kdWN0cy5waHA/Y29tbWFuZCZwcm9kdWN0aWQ=,
117 true,GET,http://192.168.1.10/cart_update.php?removep=3&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC9wcm9kdWN0cy5waHA/Y29tbWFuZCZwcm9kdWN0aWQ=,
118 true,GET,http://192.168.1.10/cart_update.php?removep=4&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC9wcm9kdWN0cy5waHA/Y29tbWFuZCZwcm9kdWN0aWQ=,
119 true,GET,http://192.168.1.10/cart_update.php?removep=5&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC9wcm9kdWN0cy5waHA/Y29tbWFuZCZwcm9kdWN0aWQ=,
120 true,GET,http://192.168.1.10/cart_update.php?removep=6&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC9wcm9kdWN0cy5waHA/Y29tbWFuZCZwcm9kdWN0aWQ=,
121 true,GET,http://192.168.1.10/view_cart.php,
122 true,GET,http://192.168.1.10/cart_update.php?emptycart=1&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC9wcm9kdWN0cy5waHA/Y29tbWFuZCZwcm9kdWN0aWQ=,
123 true,POST,http://192.168.1.10/cart_update.php,
124 true,POST,http://192.168.1.10/cart_update.php,
125 true,POST,http://192.168.1.10/cart_update.php,
```

```
125 true,POST,http://192.168.1.10/cart_update.php,
126 true,POST,http://192.168.1.10/cart_update.php,
127 true,POST,http://192.168.1.10/cart_update.php,
128 true,POST,http://192.168.1.10/cart_update.php,
129 true,GET,http://192.168.1.10/cart_update.php?removep=1&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC93YXJlaG91c2VfMS5waHA/Y29tbWFuZCZwcm9kdWN0aWQ=,
130 true,GET,http://192.168.1.10/cart_update.php?removep=2&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC93YXJlaG91c2VfMS5waHA/Y29tbWFuZCZwcm9kdWN0aWQ=,
131 true,GET,http://192.168.1.10/cart_update.php?removep=3&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC93YXJlaG91c2VfMS5waHA/Y29tbWFuZCZwcm9kdWN0aWQ=,
132 true,GET,http://192.168.1.10/cart_update.php?removep=4&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC93YXJlaG91c2VfMS5waHA/Y29tbWFuZCZwcm9kdWN0aWQ=,
133 true,GET,http://192.168.1.10/cart_update.php?removep=5&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC93YXJlaG91c2VfMS5waHA/Y29tbWFuZCZwcm9kdWN0aWQ=,
134 true,GET,http://192.168.1.10/cart_update.php?removep=6&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC93YXJlaG91c2VfMS5waHA/Y29tbWFuZCZwcm9kdWN0aWQ=,
135 true,GET,http://192.168.1.10/cart_update.php?emptycart=1&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC93YXJlaG91c2VfMS5waHA/Y29tbWFuZCZwcm9kdWN0aWQ=,
136 true,POST,http://192.168.1.10/cart_update.php,
137 true,POST,http://192.168.1.10/cart_update.php,
138 true,POST,http://192.168.1.10/cart_update.php,
139 true,GET,http://192.168.1.10/cart_update.php?removep=1&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC93YXJlaG91c2VfMi5waHA/Y29tbWFuZCZwcm9kdWN0aWQ=,
140 true,GET,http://192.168.1.10/cart_update.php?removep=2&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC93YXJlaG91c2VfMi5waHA/Y29tbWFuZCZwcm9kdWN0aWQ=,
141 true,GET,http://192.168.1.10/cart_update.php?removep=3&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC93YXJlaG91c2VfMi5waHA/Y29tbWFuZCZwcm9kdWN0aWQ=,
142 true,GET,http://192.168.1.10/cart_update.php?removep=4&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC93YXJlaG91c2VfMi5waHA/Y29tbWFuZCZwcm9kdWN0aWQ=,
143 true,GET,http://192.168.1.10/cart_update.php?removep=5&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC93YXJlaG91c2VfMi5waHA/Y29tbWFuZCZwcm9kdWN0aWQ=,
144 true,GET,http://192.168.1.10/cart_update.php?removep=6&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC93YXJlaG91c2VfMi5waHA/Y29tbWFuZCZwcm9kdWN0aWQ=,
145 true,GET,http://192.168.1.10/cart_update.php?emptycart=1&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC93YXJlaG91c2VfMi5waHA/Y29tbWFuZCZwcm9kdWN0aWQ=,
146 true,POST,http://192.168.1.10/cart_update.php,
147 true,POST,http://192.168.1.10/cart_update.php,
148 true,POST,http://192.168.1.10/cart_update.php,
149 false,GET,http://shafiul.progmaatic.com/,Out of Scope
150 true,GET,http://192.168.1.10/cart_update.php?removep=2&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC90aGFua3lvdS5waHA/aWQ9WkFQ,
151 true,GET,http://192.168.1.10/cart_update.php?removep=3&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC90aGFua3lvdS5waHA/aWQ9WkFQ,
152 true,GET,http://192.168.1.10/cart_update.php?removep=4&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC90aGFua3lvdS5waHA/aWQ9WkFQ,
153 true,GET,http://192.168.1.10/cart_update.php?removep=5&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC90aGFua3lvdS5waHA/aWQ9WkFQ,
154 true,GET,http://192.168.1.10/cart_update.php?removep=6&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC90aGFua3lvdS5waHA/aWQ9WkFQ,
155 true,GET,http://192.168.1.10/cart_update.php?emptycart=1&return_url=aHR0cDovLzE5Mi4xNjguMS4xMC90aGFua3lvdS5waHA/aWQ9WkFQ,
156
```

APPENDIX B NIKTO SCAN



APPENDIX C 192.168.1.10/ADMIN/CUSTOMERREPORT.PHP

SOMSTORE CUSTOMER REPORS
CUSTOMER REPORT
Tuesday, November 08, 2022

Employee Record: 4

Customer	Customer FullName	PHONE	Country	City	Address	Email Address
17	Steve Brown	063 89898989	Scotland	Dundee	1 Bell Street	hacklab@hacklab.com
18	Tom Jones	+252-63-418440	Scotland	Dundee	2 Brown Street	tjones@gmail.com
19	Arthur Boruc	+252-63-413844	Scotland	Aberdeen	3 White Street	Aboruc@gmail.com
20	blah	12345	Azerbaijan	Cabrayil Rayonu		Test@gmail.com

APPENDIX D EMPLOYEE TABLE

APPENDICES PART 2

APPEDNDIX E DIRBUSTER REPORT

DirBuster 1.0-RC1 - Report

http://www.owasp.org/index.php/Category:OWASP_DirBuster_Project

Report produced on Tue Nov 29 08:55:27 EST 2022

---

http://192.168.1.10:80

Directories found during testing:

Dirs found with a 403 response:

/cgi-bin/

/cgi-bin//

//cgi-bin/

/error/

/cgi-bin///

//cgi-bin//

///cgi-bin/

/error//

//error/

/cgi-bin////

////cgi-bin/

///cgi-bin//

//cgi-bin///

///error/

/error///

//error//

/error/include/

Dirs found with a 200 response:

/

/images/

/js/

/icons/

/audio/

/pictures/

/admin/images/

/images//

//

//images/

/icons//

//js/

/js//

//audio/

/audio//

/pictures//

//pictures/

/admin//images/

//admin/images/

/admin/images//

/images///

///

//images//

///images/

/js///

/icons///

///js/

//js//

//icons/

/icons/small/

///audio/

///pictures/

/pictures///

//audio//

/audio///

//pictures//

/admin///images/

/admin//images//

/admin/images///

///admin/images/

//admin//images/

//admin/images//

/admin/js/

/images////

////

////images/

//images///

///images//

/js////

////js/

/icons////

///js//

///icons/

//js///

/icons//small/

//icons//

/icons/small//

////audio/

////pictures/

///audio//

/pictures////

///pictures//

//audio///

/audio////

//pictures///

/admin///images//

/admin//images///

Dirs found with a 302 response:

/admin/

/admin//

//admin/

/admin///

///admin/

//admin//

/admin////

////admin/

Dirs found with a 401 response:

/phpmyadmin/

//phpmyadmin/

Files found during testing:

Files found with a 200 responce:

/terms.php

/home.php

/contact.php

/login.php

/index.php

/products.php

/warehouse_1.php

/warehouse_2.php

/about.php

/customer.php

/affix.php

/cart_update.php

/userValidate.php

/js/jquery-1.6.2.min.js

/js/cufon-yui.js

/js/Myriad_Pro_700.font.js

/js/jquery.jcarousel.min.js

/js/functions.js

/js/main.js

/js/DD_belatedPNG-min.js

/js/bootstrap.min.js

/js/countries.js

/js/jquery-1.9.0.min.js

/js/jquery-1.10.2.min.js

/js/jquery-1.10.2.min.map

/js/jquery.min.js

/js/moment+langs.min.js

/js/sliding.form.js

/footer.php

/extras.php

/header.php

/profile.php

//terms.php

//home.php

//about.php

//contact.php

//index.php

//products.php

//warehouse_1.php

//warehouse_2.php

//customer.php

//js/jquery-1.6.2.min.js

//js/cufon-yui.js

//js/Myriad_Pro_700.font.js

//js/jquery.jcarousel.min.js

//js/functions.js

//js/main.js

//js/DD_belatedPNG-min.js

//js/bootstrap.min.js

//js/countries.js

//js/jquery-1.9.0.min.js

//js/jquery-1.10.2.min.js

/js//DD_belatedPNG-min.js

//js/jquery-1.10.2.min.map

/js//Myriad_Pro_700.font.js

//js/jquery.min.js

//js/moment+langs.min.js

/js//bootstrap.min.js

//js/sliding.form.js

/js//countries.js

/js//cufon-yui.js

/js//functions.js

/js//jquery-1.6.2.min.js

/js//jquery-1.9.0.min.js

/js//jquery-1.10.2.min.js

/js//jquery-1.10.2.min.map

/js//jquery.jcarousel.min.js

/js//jquery.min.js

/js//main.js

/js//moment+langs.min.js

/js//sliding.form.js

//footer.php

//affix.php

//login.php

//userValidate.php

//extras.php

//header.php

//profile.php

///terms.php

///home.php

/js///DD_belatedPNG-min.js

/js///Myriad_Pro_700.font.js

/js///bootstrap.min.js

/js///countries.js

/js///cufon-yui.js

/js///functions.js

/js///jquery-1.6.2.min.js

/js///jquery-1.9.0.min.js

/js///jquery-1.10.2.min.js

/js///jquery-1.10.2.min.map

/js///jquery.jcarousel.min.js

/js///jquery.min.js

/js///main.js

/js///moment+langs.min.js

/js///sliding.form.js

///about.php

///contact.php

///index.php

///products.php

///warehouse_1.php

///warehouse_2.php

///customer.php

///js/jquery-1.6.2.min.js

///js/cufon-yui.js

///js/Myriad_Pro_700.font.js

///js/jquery.jcarousel.min.js

///js/DD_belatedPNG-min.js

///js/bootstrap.min.js

///js/functions.js

///js/countries.js

///js/main.js

///js/jquery-1.9.0.min.js

///js/jquery-1.10.2.min.js

//js//DD_belatedPNG-min.js

///js/jquery-1.10.2.min.map

//js//Myriad_Pro_700.font.js

///js/jquery.min.js

//js//bootstrap.min.js

///js/moment+langs.min.js

//js//countries.js

//js//cufon-yui.js

//js//functions.js

///js/sliding.form.js

//js//jquery-1.6.2.min.js

//js//jquery-1.9.0.min.js

//js//jquery-1.10.2.min.js

//js//jquery-1.10.2.min.map

//js//jquery.jcarousel.min.js

//js//jquery.min.js

//js//main.js

//js//moment+langs.min.js

//js//sliding.form.js

/config.php

///footer.php

///affix.php

///login.php

///userValidate.php

///extras.php

///header.php

/admin/Backup.php

/admin/OrderReports.php

/admin/EmployeeReport.php

/admin/CustomerReport.php

/admin/ProductReport.php

/admin/PaymentDelete.php

/admin/js/jquery-1.5.2.min.js

/admin/js/hideshow.js

/admin/js/jquery.tablesorter.min.js

/admin/js/jquery.equalHeight.js

///profile.php

////terms.php

////home.php

////about.php

////contact.php

/js////DD_belatedPNG-min.js

/js////Myriad_Pro_700.font.js

////index.php

/js////bootstrap.min.js

////products.php

/js////countries.js

////warehouse_1.php

/js////cufon-yui.js

////warehouse_2.php

/js////functions.js

////customer.php

/js////jquery-1.6.2.min.js

/js////jquery-1.9.0.min.js

////js/jquery-1.6.2.min.js

/js////jquery-1.10.2.min.js

////js/cufon-yui.js

/js////jquery-1.10.2.min.map

////js/Myriad_Pro_700.font.js

/js////jquery.jcarousel.min.js

////js/DD_belatedPNG-min.js

////js/jquery.jcarousel.min.js

/js////jquery.min.js

////js/bootstrap.min.js

////js/functions.js

/js////main.js

////js/main.js

/js////moment+langs.min.js

////js/countries.js

/js////sliding.form.js

////js/jquery-1.9.0.min.js

////js/jquery-1.10.2.min.js

////js/jquery-1.10.2.min.map

////js/jquery.min.js

////js/moment+langs.min.js

////js/sliding.form.js

///js//DD_belatedPNG-min.js

///js//Myriad_Pro_700.font.js

///js//bootstrap.min.js

///js//countries.js

///js//cufon-yui.js

///js//functions.js

///js//jquery-1.6.2.min.js

///js//jquery-1.9.0.min.js

///js//jquery-1.10.2.min.js

///js//jquery-1.10.2.min.map

///js//jquery.jcarousel.min.js

///js//jquery.min.js

///js//main.js

///js//moment+langs.min.js

///js//sliding.form.js

//js///DD_belatedPNG-min.js

//js///Myriad_Pro_700.font.js

//js///bootstrap.min.js

//js///countries.js

//js///cufon-yui.js

//js///functions.js

//js///jquery-1.6.2.min.js

//js///jquery-1.9.0.min.js

//js///jquery-1.10.2.min.js

//js///jquery-1.10.2.min.map

//js///jquery.jcarousel.min.js

//js///jquery.min.js

//js///main.js

//js///moment+langs.min.js

//js///sliding.form.js

//config.php

////footer.php

////affix.php

////login.php

////userValidate.php

////extras.php

////header.php

Files found with a 302 responce:

/feedback_process.php

/employeeValidate.php

//feedback_process.php

/admin/index.php

//employeeValidate.php

/admin//index.php

///feedback_process.php

//admin/index.php

///employeeValidate.php

/admin/order.php

/admin/add_warehouse.php

/admin/add_product.php

/admin/Employee.php

/admin/add_category.php

/admin/customerTable.php

/logout.php

/admin/searchprod.php

/admin///index.php

////feedback_process.php

///admin/index.php

//admin//index.php

////employeeValidate.php

/admin//order.php

/admin//add_warehouse.php

/admin//add_product.php

Files found with a 500 responce:

/preview.php

//preview.php

///preview.php